

INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Introducción a la Computación 22.06

Modos de direccionamiento, Introducción a la Programación, Stack y Subrutinas

Trabajo de Laboratorio N ° 02

Curso 2020A – 1er Cuatrimestre

GRUPO N°: 04		INTEGRANTES
Legajo N°	Nombre	
62490	Marcos Dario Alegre	
62012	Santiago Feldman	
62447	Anna Candela Gioia Perez	
61608	Sergio Tomás Rojo Gomes	
61758	Juan Sebastián Helou	

	Fecha	Docente
Realizado	15/09/2020	
Presentado	16/09/2020	
Aprobado		

Introducción a la computación

Trabajo práctico N° 2

Modos de Direcccionamiento

Ejercicio N°1

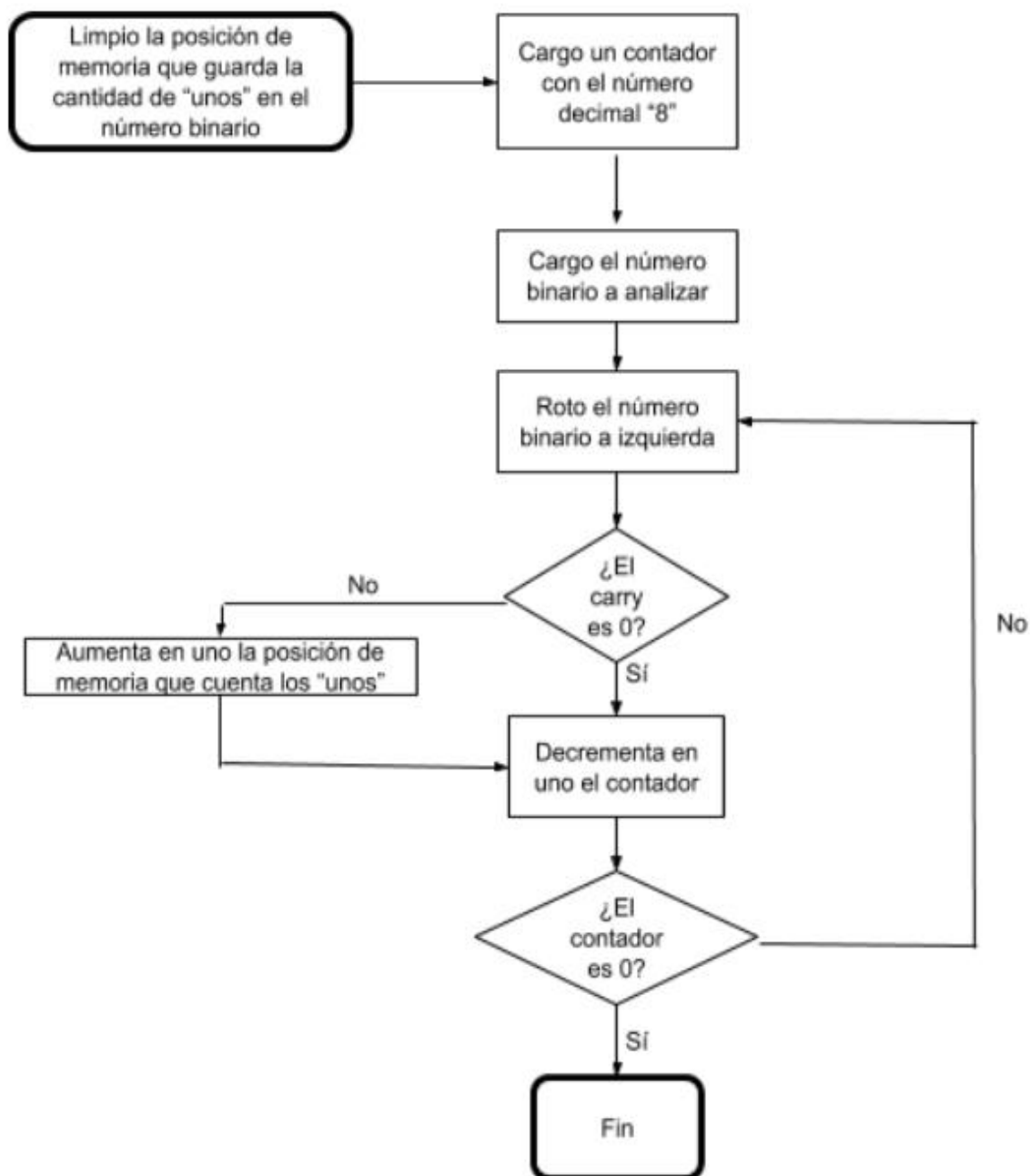
Se pide:

a) Completar la siguiente tabla

Address	OpCode	Label	Mnemónico	Operación	Veces	Ciclos	Duración (µseg)
2000	7F 00 20		clr \$0020	Limpia el byte de memoria	1	6	3
2003	86 08		ldaa #08	Carga el registro A con 8 decimal	1	2	1
2005	C6 D2		ldab #11010010b	Carga el registro B con 208 en binario	1	2	1
2007	59	L01	rolb	Rota B a la izquierda	8	2	8
2008	24 03		bcc L00	Si el carry es 0 entonces va a L00	8	3	12
200A	7C 00 20		inc \$0020	Aumenta en 1 la posición de memoria \$0020	8	6	24
200D	4A	L00	deca	Decrementa el registro A	8	2	8
200E	26 F7		bne L01	Va a L01 si Z es 0	8	3	12
2010	3F		swi	fin	1	14	7

Total 76

b) Realizar el diagrama de flujo correspondiente



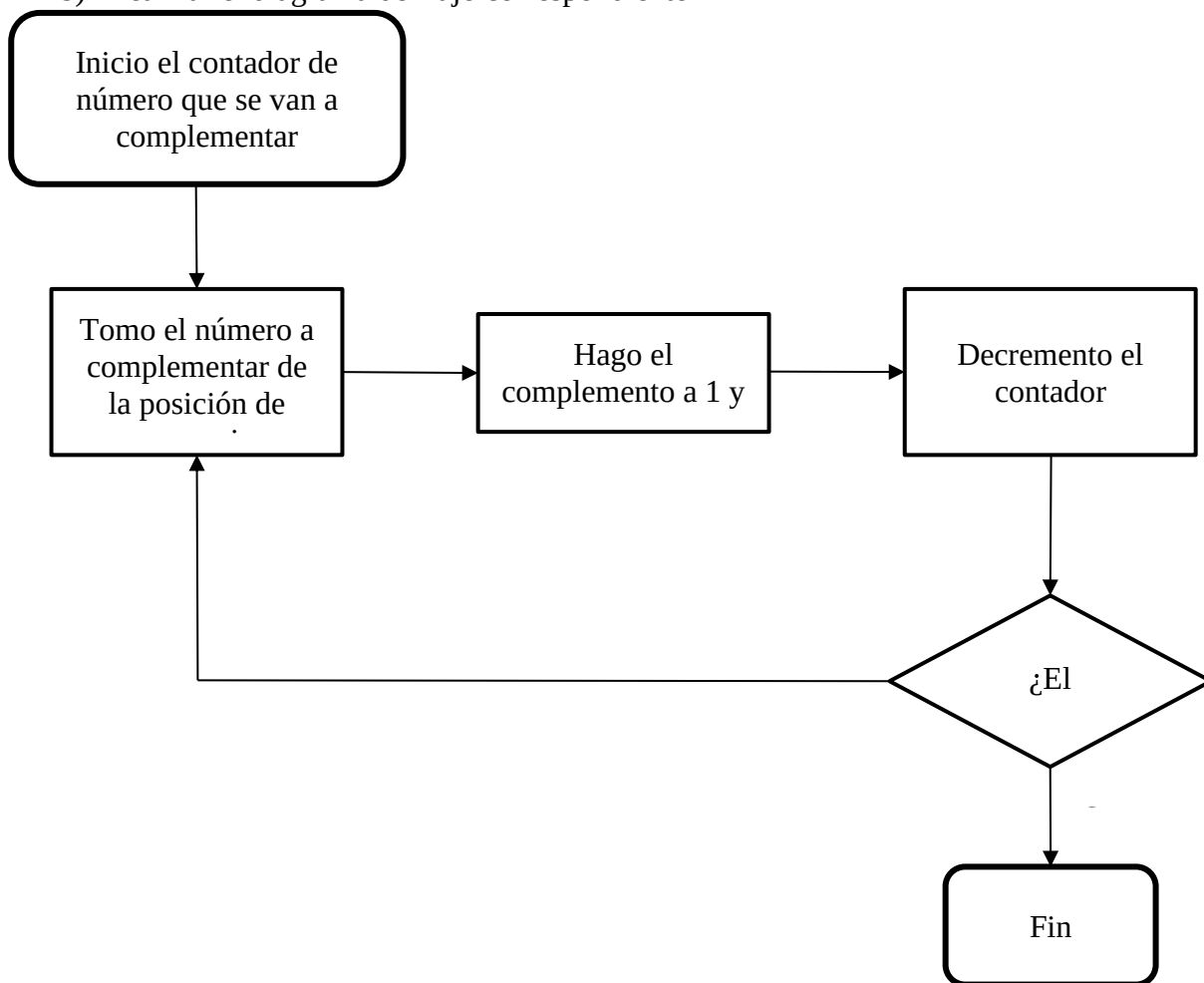
c) Describir qué hace el programa y de ser posible simularlo.

El programa analiza cuántos “unos” hay en un número binario y guarda el resultado en una posición de memoria especificada.

Ejercicio N°2

Se pide:

- a) Completar la siguiente tabla
- b) Realizar el diagrama de flujo correspondiente



- c) Describir qué hace el programa y de ser posible simularlo.

La función del programa es realizar los complementos a 1 de los valores que se encuentran en las posiciones de memoria de \$0060 a \$0064 y colocar esos complementos respectivamente en las posiciones de memoria de \$0050 a \$0054.

Address	Op. Code	Label	Nemónico	Operación	Veces	Ciclos	Duración (µseg)
2000	18 CE 00 50		ldy #\$0050	Carga el número 0050 en hexadecimal en el registro Y	1	4	2
2004	CE 00 60		ldx #\$0060	Carga el número 0060 en hexadecimal en el registro X	1	3	1,5
2007	C6 05		ldab #5	Carga el número 5 en decimal en el registro AccB	1	2	1
2009	A6 00	L01	ldaa 00,x	Carga el registro AccA con el número que se encuentra en la celda de la posición de memoria que marca el registro X	5	4	10
200B	43		coma	Hace el complemento a 1 del número que está cargado en el registro AccA	5	2	5
200C	18 A7 00		staa 00,y	Carga el número que está cargado en el registro AccA en la posición de memoria que marca el registro Y	5	5	12,5
200F	08		inx	Incrementa el número que está cargado en el registro X	5	3	7,5
2010	18 08		iny	Incrementa sumando 1 al número que está cargado en el registro Y	5	4	10
2012	5A		decb	Decrementa restando 1 al número cargado en el registro AccB	5	2	5
2013	26 FA		bne L01	Ramificar a L01 si el número cargado en el registro AccB no es 0 (Z=0)	5	3	7,5
2015	3F		swi	Si el número cargado en el registro AccB es 0 (Z=1) interrumpe el Software	1	14	7

Total

69

d) Teniendo en cuenta los valores iniciales en memoria, completar los valores finales luego de la ejecución en la tabla siguiente.

Antes de la ejecución

0050	0051	0052	0053	0054	0055	0056
12	33	41	55	01	0A	38
0060	0061	0062	0063	0064	0065	0066
32	A3	B1	C0	43	20	A2

Después de la ejecución

0050	0051	0052	0053	0054	0055	0056
CD	5C	4E	3F	BC	0A	38
0060	0061	0062	0063	0064	0065	0066
32	A3	B1	C0	43	20	A2

Ejercicio N° 3

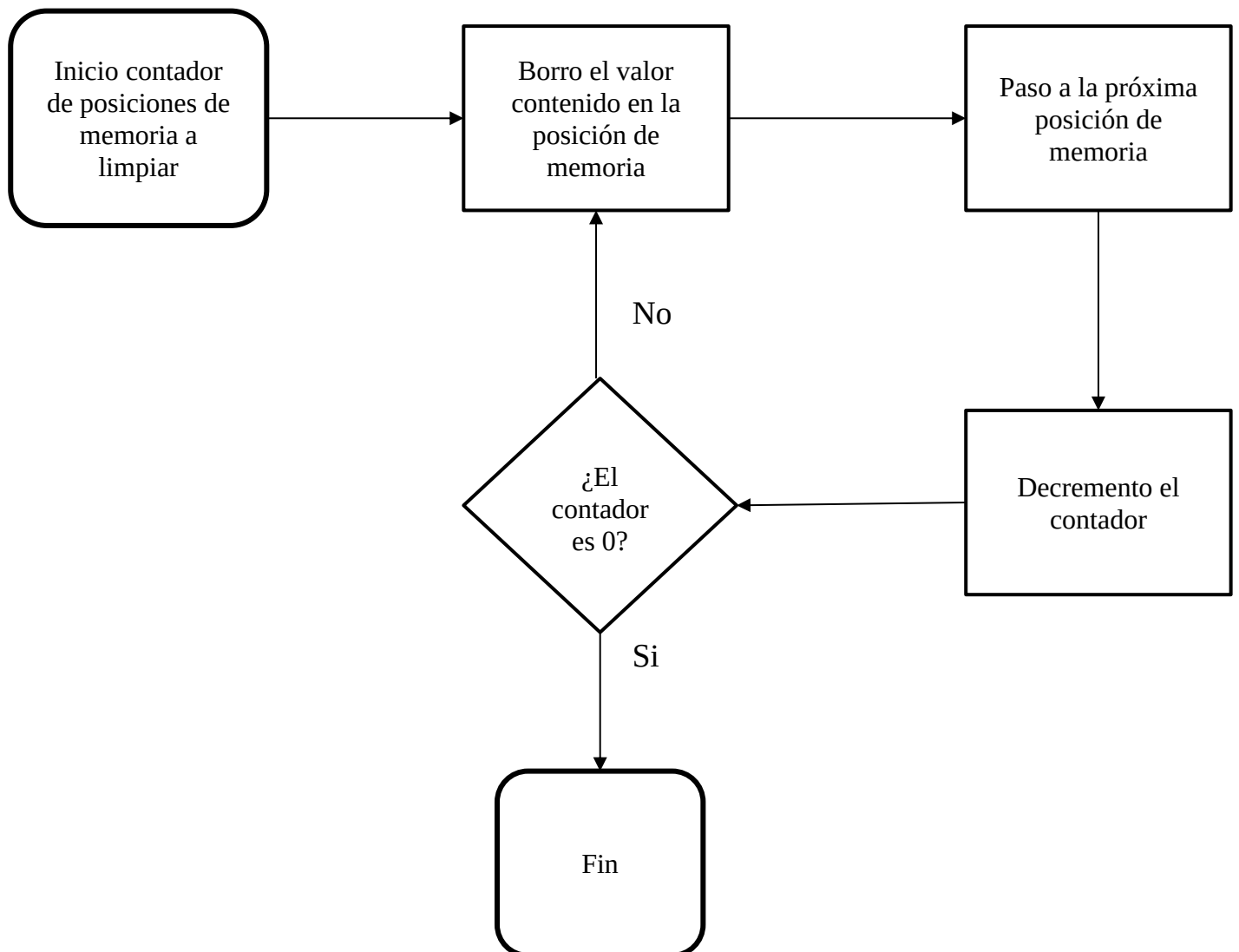
Dado el siguiente programa se pide:

- Completar la siguiente tabla
- Hacer el diagrama de flujo

Address	OpCode	Label	Mnemónico	Operación.	Ciclos	Veces	Duración (useg)
0000	86 03		ldaa #3	Carga el número \$03 al AccA	2	1	1
0002	CE 30 00		ldx #\$3000	Carga el número \$3000 al registro X	3	1	1,5
0005	6F 00	L01	clr 0,X	Limpia la posición de memoria a la que el registro X apunta	6	3	9
0007	08		inx	Incrementa el valor del registro X en 1	3	3	4,5
0008	4A		deca	Decrementa el valor del AccA	2	3	3
0009	26 FA		bne L01	Ramificar a L01 si el número cargado en el registro AccA es distinto a 0 (Z=0)	3	3	4,5
000B	3F		swi (STOP)	Si el número en AccA es 0 (Z=1), interrumpe el Software	14	1	7

Total

30,5



d) Completar la siguiente tabla ejecutando paso a paso el programa y probarlo en el simulador.

	Instrucción	Memoria				CCR				Registros				
		\$3000	\$3001	\$3002	\$3003	N	Z	V	C	A	B	X	Y	PC
-----	Estado Inicial	01	22	AA	05	0	0	0	0	3A	32	2400	2401	0000
0000	ldaa #3	01	22	AA	05	0	0	0	0	03	32	2400	2401	0002
0002	ldx #\$3000	01	22	AA	05	0	0	0	0	03	32	3000	2401	0005
0005	clr 0,x	00	22	AA	05	0	1	0	0	03	32	3000	2401	0007
0007	inx	00	22	AA	05	0	0	0	0	03	32	3001	2401	0008
0008	deca	00	22	AA	05	0	0	0	0	02	32	3001	2401	0009
0009	bne	00	22	AA	05	0	0	0	0	02	32	3001	2401	0005
0005	cl 0,x	00	00	AA	05	0	1	0	0	02	32	3001	2401	0007
0007	inx	00	00	AA	05	0	0	0	0	02	32	3002	2401	0008
0008	deca	00	00	AA	05	0	0	0	0	01	32	3002	2401	0009
0009	bne	00	00	AA	05	0	0	0	0	01	32	3002	2401	0005
0005	clr 0,x	00	00	00	05	0	1	0	0	01	32	3002	2401	0007
0007	inx	00	00	00	05	0	0	0	0	01	32	3003	2401	0008
0008	deca	00	00	00	05	0	1	0	0	00	32	3003	2401	0009
0009	bne	00	00	00	05	0	1	0	0	00	32	3003	2401	000B
000B	swi	00	00	00	05	0	1	0	0	00	32	3003	2401	-----

Asumir que la duración del ciclo de maquina es de 0.5 µseg.

Introducción a la Programación

Ejercicio N° 4

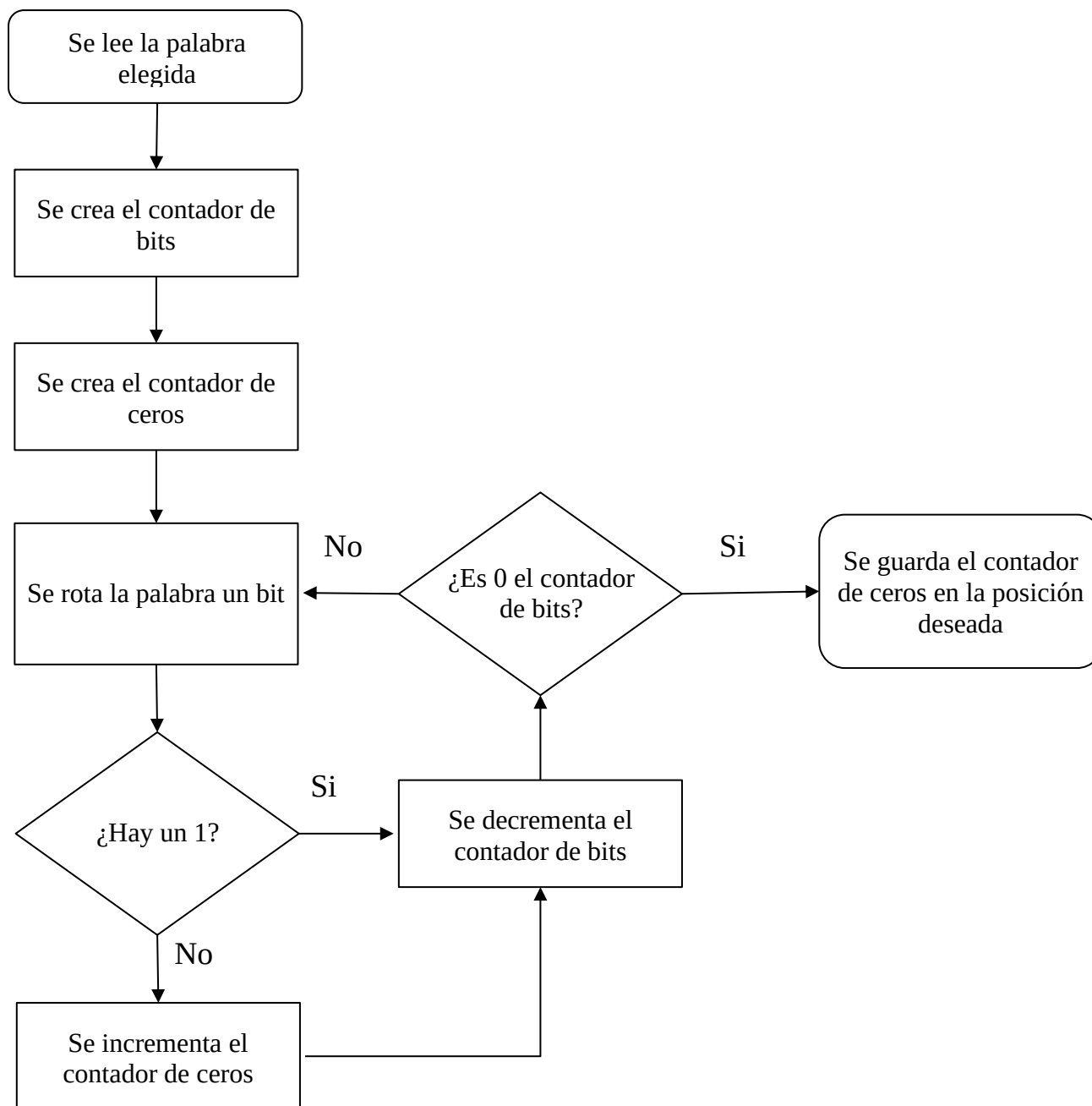
Realice un programa que cuenta la cantidad de 0's que contiene la palabra ubicada en la posición de memoria \$2018. El resultado de la cuenta se deberá colocar en la posición de memoria \$2019.

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           12/Sep/20  21:34:29  #
#
#   Source   =   ej4tp2.msa                               #
#   List     =   ej4tp2.lst                               #
#   Object   =   ej4tp2.r07                               #
#   Options  =                                           #
#
#
#                                           (c) Copyright IAR Systems 1990 #
#####
```



```
1 0000          p68h11          ;indica el modelo del microprocesador
2
3 2000          org      $2000
4 2000 B62018    ldaa      $2018      ;carga la palabra elegida en el registro a
5 2003 C608      ldab      #8
6 2005 F73000    stab      $3000      ;se crea el contador de bits
7 2008 5F        clrb
8 2009 48        loop    asla        ;se crea el contador de ceros
9 200A 2501      bcs      decount     ;si hay un 1 en el bit decrementa el contador de bits
10 200C 5C       incb
11 200D 7A3000   decount dec      $3000 ;si hay un 0 incrementa el contador de ceros
12 2010 26F7     bne      loop
13 2012 F72019   store    stab      $2019 ;si el contador de bits no es 0 pasa al siguiente bit
14 2015 7E2015   fin      jmp      fin ;si es cero guarda la cantidad de ceros contados
15
16 2018          END

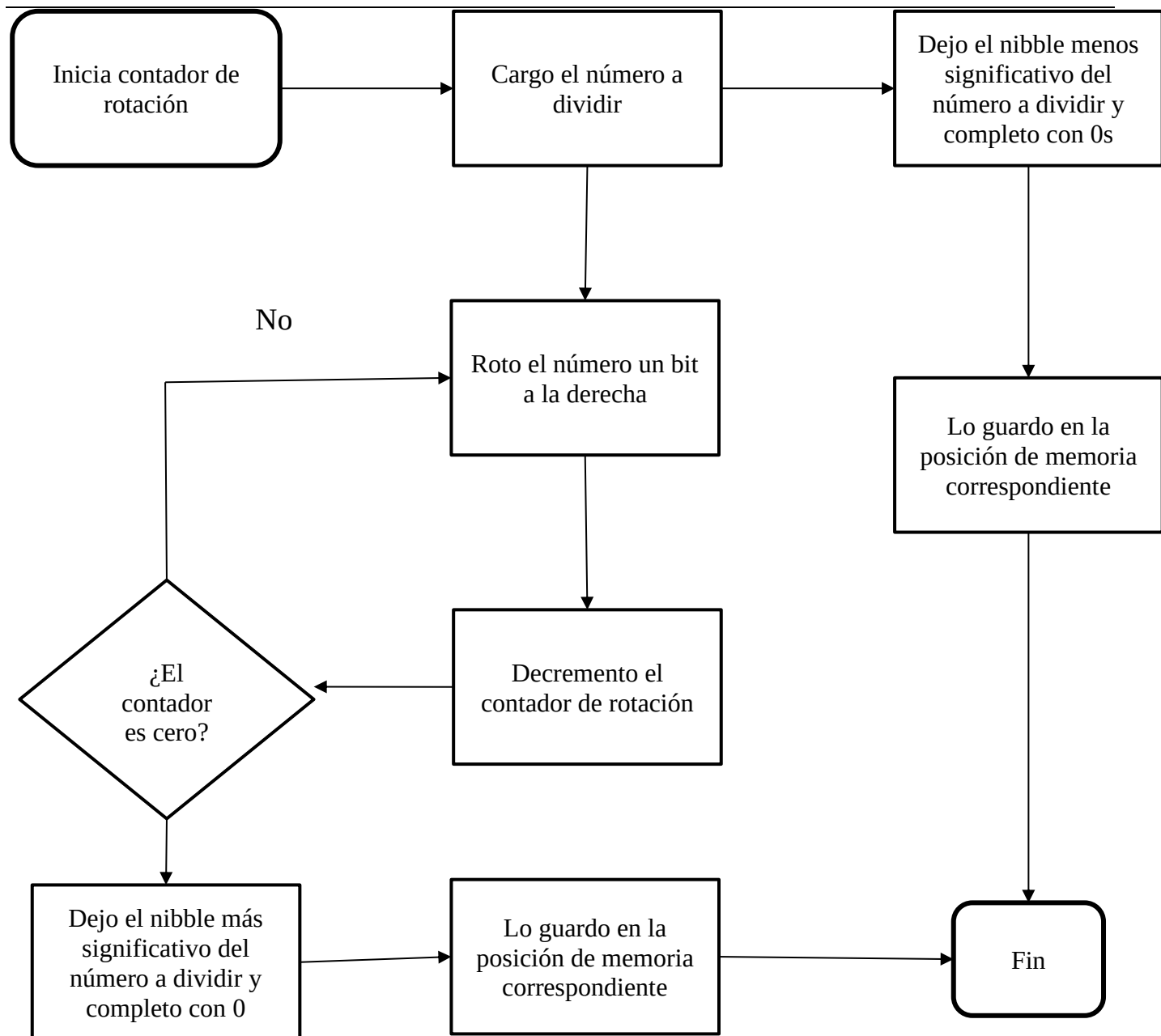
Errors:  None      #####
Bytes:   24        # ej4tp2 #
CRC:     B8A9      #####
```



Ejercicio N° 5

Divida el contenido de la posición de memoria \$2200 en dos secciones de 4 bits (nibble). Colocar el nibble más significativo en la posición \$2202 y el nibble menos significativo en la posición \$2203. Completar con ceros las palabras para formar el byte.

```
#####  
#                                                                 #  
#   Micro Series 6801 Assembler v2.00/bos           14/Sep/20  18:56:00  #  
#                                                                 #  
#       Source   =   tp2n5pg.msa                               #  
#       List     =   tp2n5pg.lst                               #  
#       Object   =   tp2n5pg.r07                               #  
#       Options  =                                           #  
#                                                                 #  
#                                     (c) Copyright IAR Systems 1990 #  
#####  
  
1  0000                p68H11                ;declara el tipo de microprocesador  
2  2210                org      $2210  
3  2210 C604           ldab    #4              ;inicia el contador de rotación  
4  2212 F72209         stab    $2209          ;paso el contador a esa posición de memoria  
5  2215 B62200         ldaa    $2200          ;carga el número a dividir  
6  2218 F62200         ldab    $2200          ;carga el número a dividir  
7  000F      mascara EQU  00001111b          ;se usa para hacer AND con los nibbles  
8  221B 840F          anda    #mascara        ;deja sólo el nibble menos significativo y completa con 0s  
9  221D 56            repito  rorb            ;roto el nibble más significativo a la parte del menos  
10 221E 7A2209        dec     $2209           ;decremento el contador  
11 2221 26FA          bne     repito          ;si no es 0 el contador vuelvo a rotar el número  
12 2223 C40F          andb    #mascara        ;deja sólo el nibble más significativo y completa con 0s  
13 2225 B72203        staa    $2203          ;guarda el nibble menos significativo  
14 2228 F72202        stab    $2202          ;guarda el nibble más significativo  
15 222B 7E222B      fin     jmp     fin  
16 222E                END  
  
Errors:  None          #####  
Bytes:   30            # tp2n5pg #  
CRC:     EE25          #####
```



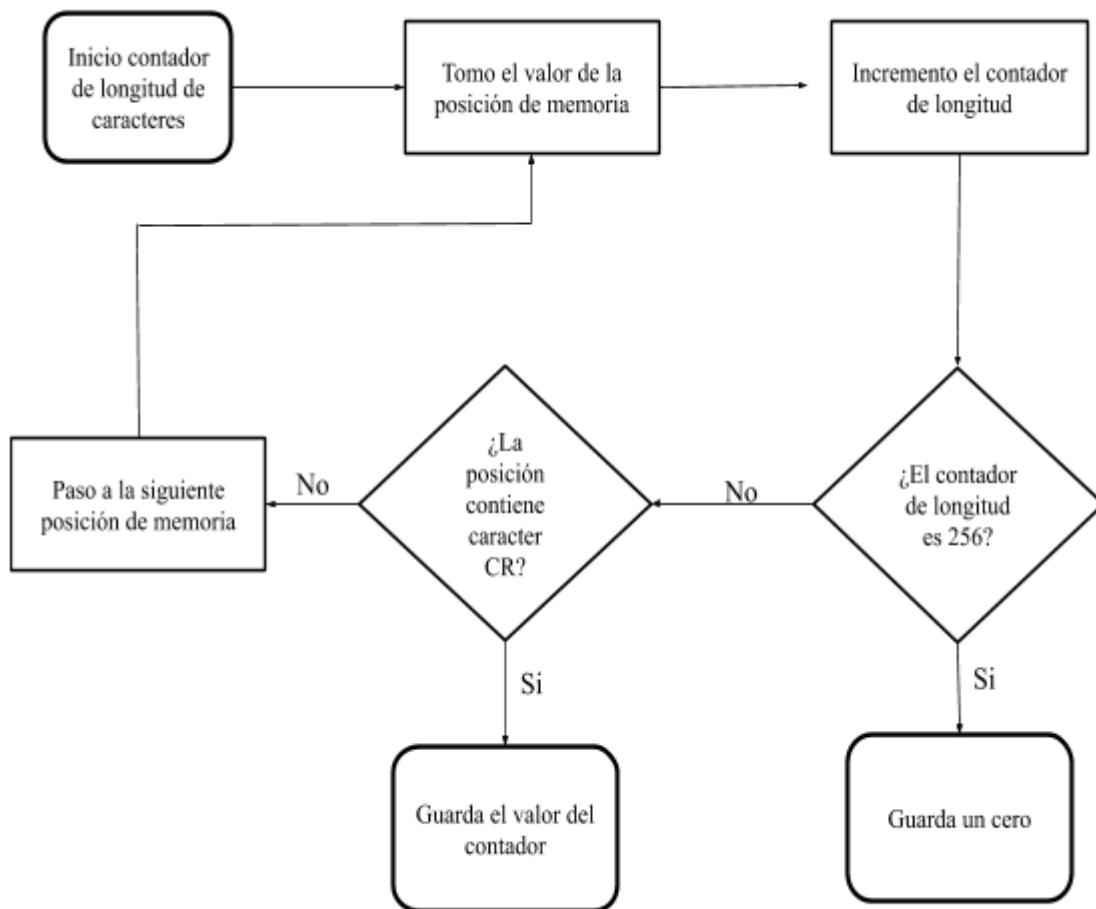
Ejercicio N°6

Determinar la longitud de una cadena de caracteres. La misma comienza en la dirección \$2070 y el final de la misma está determinada por el carácter ASCII “CR” (equivalente hexadecimal \$0D). Guardar la longitud en la dirección \$2000 (asumir que longitud máxima puede ser 255)

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           12/Sep/20  22:39:08  #
#
#   Source   =   ej6tp2.msa                                #
#   List     =   ej6tp2.lst                                #
#   Object   =   ej6tp2.r07                                #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1  0000                p68h11
2
3  3000                org      $3000
4  3000 CE2070         ldx      #$2070 ;se apunta el registro x a la dirección deseada
5  3003 5F            clrb
6  3004 A600      loop    ldaa    0,x
7  3006 5C                incb          ;se aumenta el contador de caracteres
8  3007 C100         cmpb    #$00 ;si se llega a 255 caracteres y ninguno contiene CR guarda un 0
9  3009 2708         beq     store ;pues no existe una cadena menor de 256 caracteres que contenga CR
10 300B 810D         cmpa    #$0D ;se fija si el caracter que esta leyendo es CR
11 300D 2704         beq     store ;si es CR pasa a guardar la longitud de la cadena
12 300F 08            inx          ;si no es pasa al siguiente
13 3010 7E3004        jmp     loop
14 3013 F72000      store    stab  $2000
15 3016 7E3016      fin      jmp    fin
16
17 3019                END

Errors:  None          #####
Bytes:   25            # ej6tp2 #
CRC:     8C82          #####
```



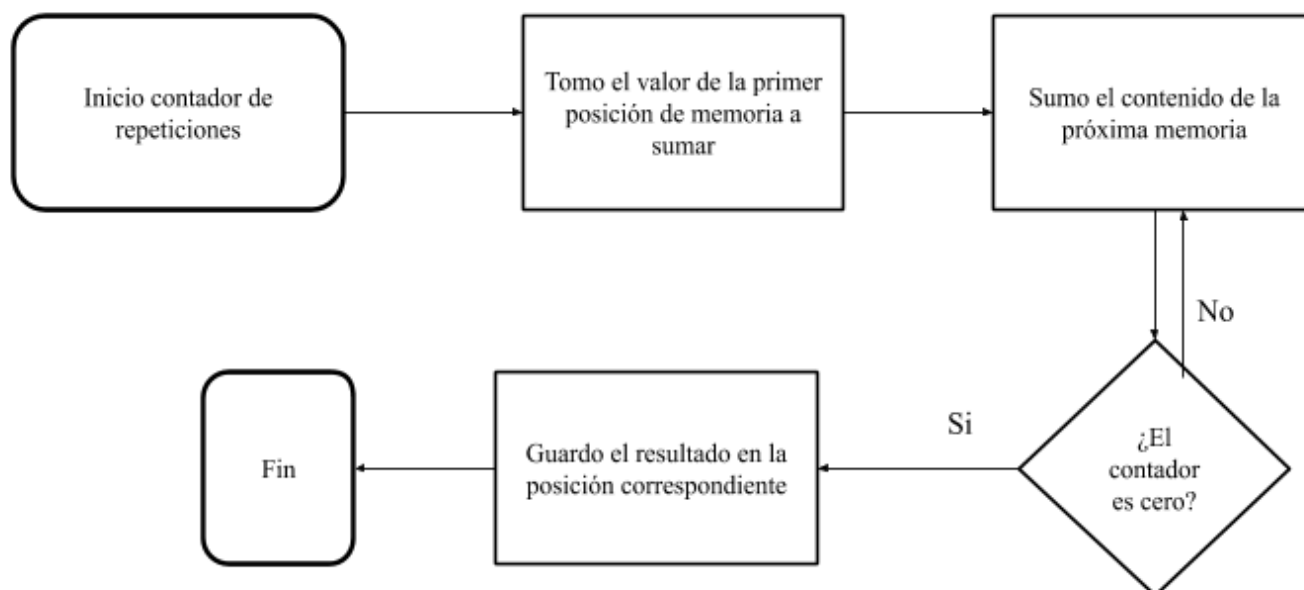
Ejercicio N°7

Escribir un programa que sume el contenido de las posiciones de memorias comprendidas entre \$5000 y \$5010 y guarde el resultado a partir de la dirección \$3000.

```
#####
#
#   Micro Series 6801 Assembler v2.00/DOS           15/Sep/20  17:07:20   #
#
#   Source = tp2n7h.msa                                     #
#   List   =  tp2n7h.lst                                     #
#   Object =  tp2n7h.r07                                     #
#   Options =                                              #
#
#                                                    (c) Copyright IAR Systems 1990 #
#####

1  0000                p68H11                ;declara el microprocesador a usar
2  3002                org      $3002
3  3002 8611            ldaa   #17              ;inicia contador de repeticiones
4  3004 18CE5000        ldy    #$5000
5  3008 18E600    repito  ldab   0,y          ;tomo el valor de la memoria a sumar
6  300B 3A          abx
7  300C 1808        iny              ;apunta a la siguiente posición de memoria a sumar
8  300E 4A          deca              ;decrementa el contador
9  300F 26F7        bne    repito ;si el contador no es 0 repito
10 3011 FF3000       stx    $3000 ;si el contador es 0 guarda la solución
11 3014 7E3014    fin    jmp    fin
12 3017                END

Errors:  None          #####
Bytes:   21            # tp2n7h #
CRC:     96B2          #####
```



Stack y Subrutinas

Ejercicio N°8

Escribir una subrutina llamada **strcpy** (**string copy**), la misma deberá copiar un string (cadena de caracteres) que se encuentra en una zona de memoria *origen* a otra zona de memoria *destino*.

La subrutina recibe como parámetro un puntero (dirección de memoria) a la dirección de inicio del string en el registro IX y la dirección destino en el registro IY. Usar la directiva FCC y FCB para definir la cadena origen de prueba y RMB para reservar la zona de memoria destino (reservar por defecto 20 bytes). El string origen tiene como terminador el cero. La copia termina una vez copiado el terminador \$00. Si la función puede realizar la copia exitosamente deberá colocar un cero en el registro A, de no hacerlo deberá colocar el mismo con un uno.

Se deberán emplear menos de 20 instrucciones.


```
#####
#
#   Micro Series 6801 Assembler v2.00/bOS           15/Sep/20  13:04:36   #
#
#   Source   =   ej8p4.msa
#   List     =   ej8p4.lst
#   Object   =   ej8p4.r07
#   Options  =
#
#                                           (c) Copyright IAR Systems 1990 #
#####

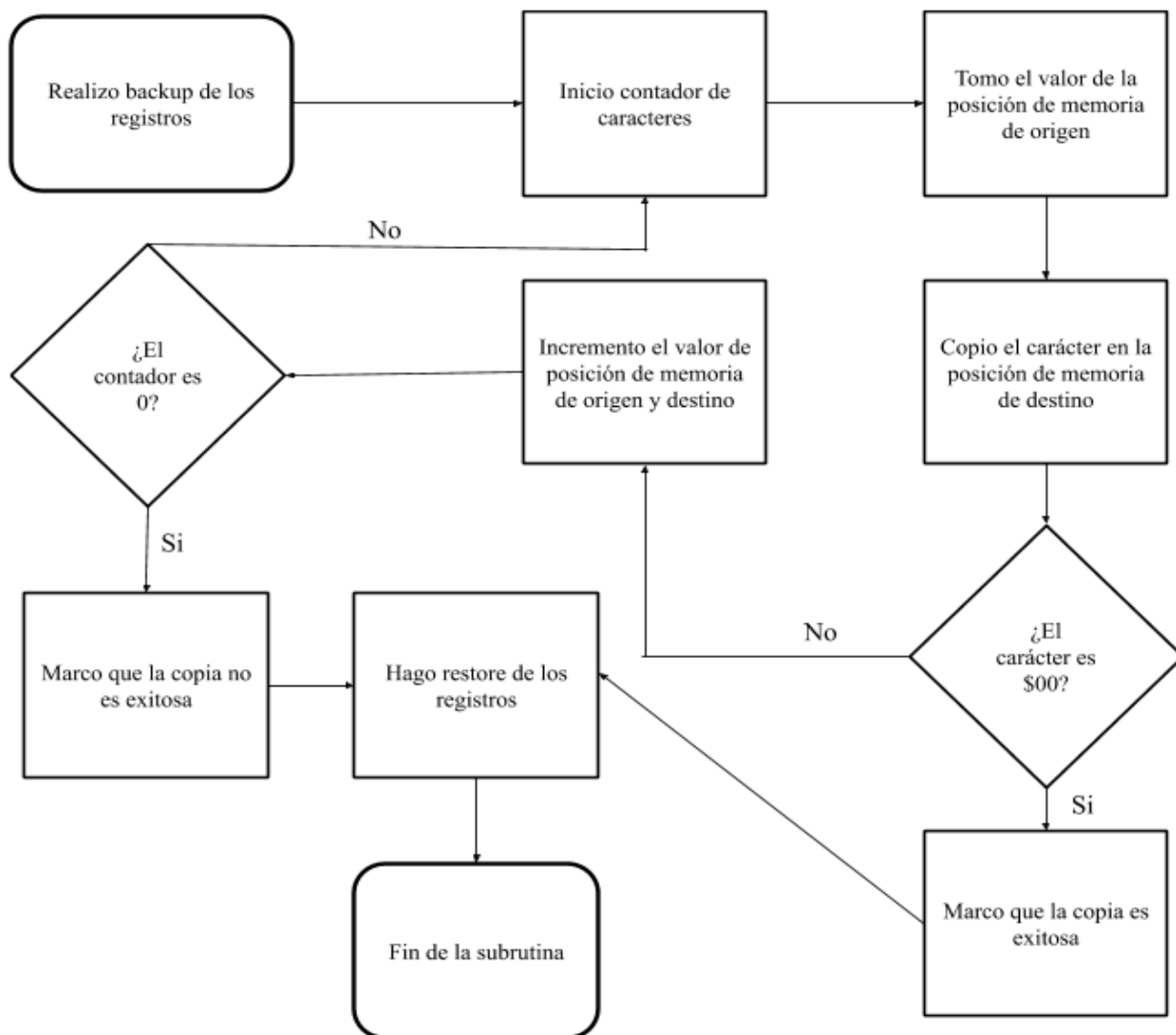
1
2      *****
3      *                               EJERCICIO 8                               *
4      *   El programa se encarga de copiar un string menor o                 *
5      *   igual a 20 caracteres de una zona de memoria a otra                *
6      *   *****
7
8
9 0000          p68h11          ;declara el microprocesador
10
11
12      *****
13      *                               MAIN PROGRAM                               *
14      *   Llama a la subrutina strcpy para copiar la cadena de               *
15      *   caracteres que se encuentran en la zona de origen a                *
16      *   la que apunta el registro IX en la zona de destino                *
17      *   a la que apunta el registro IY                                     *
18      *   *****
19
20 4000          org      $4000
21 4000          main    equ      *
22 4000 8E450E      lds     #stackp      ;necesario para el stack y subrutina
23 4003 CE3000      ldx     #origen      ;carga la posición de memoria a leer
24 4006 18CE3100     ldy     #destino     ;carga la posición de memoria en donde se copia
25 400A BD4200      jsr     strcpy      ;llama a la subrutina
26 400D 7E400D      fin     jmp     fin
27      *****
28      *                               SUBROUTINA STRCPY                               *
29      *   Las subrutina copia la cadena de caracteres que se                 *
30      *   encuentran en la zona de origen a la que apunta el                 *
31      *   registro IX en la zona de destino a la que apunta                  *
32      *   el registro IY
33      *   *****
34
35
36 4200          org      $4200
37 4200          strcpy  equ      *
38 4200 183C        pshy          ;hace backup de los registros
39 4202 3C          pshx
40 4203 37          pshb
41 4204 C614        ldab     #20          ;inicia contador de caracteres
42 4206 A600          repito  ldaa     0,x      ;lee el carácter la posición de memoria de origen
43 4208 18A700        staa     0,y      ;copia el carácter en la posición de memoria de destino
44 420B 270B          beq     termino      ;si el carácter es 0 entonces termina de leer y copiar
45 420D 08           inx          ;pasa a la siguiente posición de memoria de origen
46 420E 1808          iny          ;pasa a la siguiente posición de memoria de destino
47 4210 5A           decb          ;decrementa el contador de caracteres
48 4211 26F3          bne     repito      ;si el contador no es 0 repite el proceso
49 4213 8601          ldaa     #1          ;si el contador es 0, la copia no fue exitosa
50 4215 7E421E        jmp     fins
51 4218 8600          termino  ldaa     #0          ;la copia fue exitosa porque se encontró el fin del string
```

```

52 421A 33          pulb          ;dentro de los 20 caracteres
53 421B 38          pulx          ;hace restore de los registro
54 421C 1838        puly
55 421E 39          fins      rts          ;termina la subrutina
56
57
58 *****
59 *                AREA DE DATOS                *
60 *      Lista de los caracteres ASCII que se van a copiar      *
61 *      y el comienzo del origeny destino y cantidad máxima de *
62 *      caracteres que se pueden copiar                    *
63 *****
64
65
66 3000          origen equ      $3000          ;define el comienzo de donde se va a leer el string
67 3100          destino equ     $3100          ;define el comienzo de donde se va a copiar el string
68 3100          org      destino
69 3100          rmb      20          ;define la cantidad máxima de caracteres que puede copiar
70 3000          org      origen
71 3000 696E7472   fcc      'introduccion a la computacion' ;cadena de caracteres que se quiere copiar
    3004 6F647563
    3008 63696F6E
    300C 2061206C
    3010 6120636F
    3014 6D707574
    3018 6163696F
    301C 6E
72 301D 00          fcb      $00          ;define la terminación del string
73
74
75 *****
76 *                AREA DE STACK                *
77 *      Se necesitan 5 posiciones, se deja un margen extra      *
78 *****
79
80 4500          org      $4500
81 4500          stack   rmb      15
82 450E          stackp  equ      *-1
83
84
85 450F          END

Errors:  None          #####
Bytes:   77          # ej8p4 #
CRC:    8948          #####

```



Ejercicio N°9

Repetir el ejercicio 8, esta vez **strcpy** recibe por stack las direcciones origen y destino. Se respetan las mismas condiciones para la reserva de memoria y definición de las cadenas. Se modifica el registro de error. Si el programa no puede copiar exitosamente la cadena deberá colocar el bit de carry en uno, caso contrario deberá quedar en cero. **Se deberán emplear menos de 20 instrucciones.**

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           15/Sep/20   20:21:03   #
#
#   Source   =   ej9p5.msa                               #
#   List     =   ej9p5.lst                               #
#   Object   =   ej9p5.r07                              #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####
```

```
1
2   *****
3   *                               EJERCICIO 9                               *
4   *   El programa se encarga de copiar un string menor o                 *
5   *   igual a 20 caracteres de una zona de memoria a otra                 *
6   *   *****
7
8
9   0000                p68h11                ;declara el microprocesador
10
11
12   *****
13   *                               MAIN PROGRAM                               *
14   *   Llama a la subrutina strcpy para copiar la cadena de                 *
15   *   caracteres que se encuentran en la zona de origen a                 *
16   *   en la zona de destino                                               *
17   *   *****
18
19   4000                org    $4000
20   4000                main   equ    *
21   4000 8E4513        lds    #stackp                ;necesario para el stack y subrutina
```

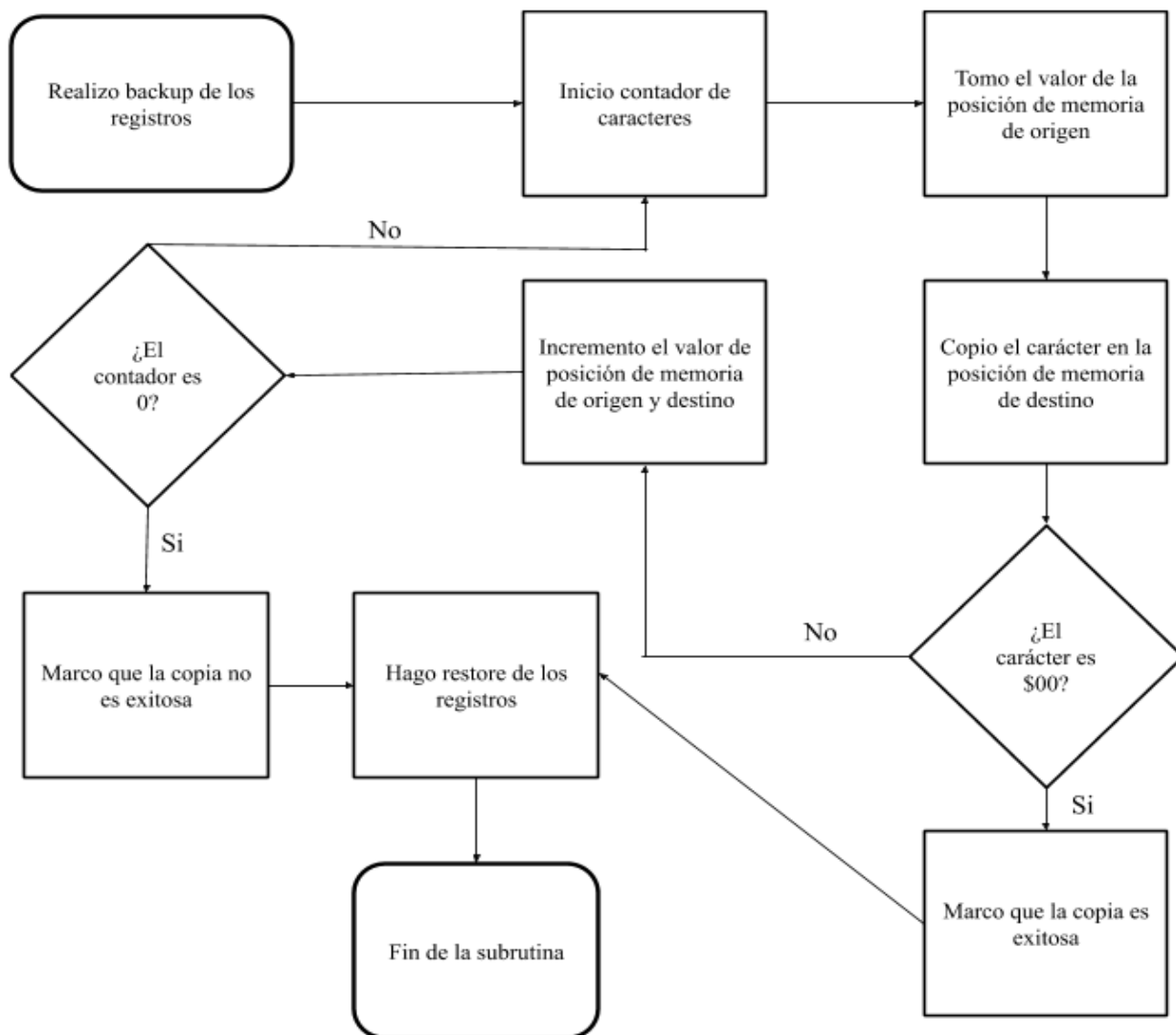
```

22 4003 CE3000      ldx    #origen      ;carga la posición de memoria a leer
23 4006 18CE3100    ldy    #destino    ;carga la posición de memoria en donde se copia
24 400A 183C        pshy                ;hace backup de los registros y
25 400C 3C          pshx                ;reserva posiciones de stack
26 400D BD4200      jsr     strcpy      ;llama a la subrutina
27 4010 38          pulx
28 4011 1838        puly                ;hace restore de los registros
29 4013 7E4013      fin    jmp     fin
30
31                  *****
32                  *                SUBROUTINA STRCPY                *
33                  *      Debe recibir por stack la dirección del origen y      *
34                  *      destino y devuelve la copia del string              *
35                  *****
36
37
38 4200              org     $4200
39 4200      strcpy    equ     *
40 4200 37           pshb                ;hace backup de los registros
41 4201 30           tsx                ;crea frame pointer
42 4202 1AEE05       ldy     5,x         ;toma el valor de destino
43 4205 EE03         ldx     3,x         ;toma el valor de origen
44 4207 C614         ldab    #20         ;inicia contador de caracteres
45 4209 A600      repito ldaa    0,x         ;lee el carácter la posición de memoria de origen
46 420B 18A700       staa    0,y         ;copia el carácter en la posición de memoria de destino
47 420E 270A         beq     termino     ;si el carácter es 0 entonces termina de leer y copiar
48 4210 08          inx                ;pasa a la siguiente posición de memoria de origen
49 4211 1808         iny                ;pasa a la siguiente posición de memoria de destino
50 4213 5A          decb                ;decrementa el contador de caracteres
51 4214 26F3         bne     repito      ;si el contador no es 0 repite el proceso
52 4216 0D          sec                ;si el contador es 0, la copia no fue exitosa
53 4217 7E421C       jmp     fins
54 421A 0C          termino clc    #0      ;la copia fue exitosa porque se encontro el fin del string
55 421B 33          pulb                ;hace restore de los registro
56 421C 39          fins    rts          ;termina la subrutina
57
58
59                  *****
60                  *                AREA DE DATOS                *
61                  *      Lista de los caracteres ASCII que se van a copiar      *
62                  *      y el comienzo del origen y destino y cantidad máxima de *
63                  *      caracteres que se pueden copiar                      *
64                  *****
65
66
67 3000      origen    equ     $3000      ;define el comienzo de donde se va a leer el string
68 3100      destino   equ     $3100      ;define el comienzo de donde se va a copiar el string
69 3100              org     destino
70 3100              rmb     20          ;define la cantidad máxima de caracteres que puede copiar
71 3000              org     origen
72 3000 696E7472      fcc     'introduccion a la computacion' ;cadena de caracteres que se quiere copiar

```

```
3004 6F647563
3008 63696F6E
300C 2061206C
3010 6120636F
3014 6D707574
3018 6163696F
301C 6E
73 301D 00          fcb      $00          ;define la terminación del string
74
75
76                *****
77                *              AREA DE STACK              *
78                *      Se necesitan 5 posiciones, se deja un margen extra      *
79                *****
80
81 4500                org      $4500
82 4500                stack   rmb    20
83 4513                stackp  equ    *-1
84
85
86 4514                END

Errors:  None          #####
Bytes:   81            # ej9p5 #
CRC:    0A41          #####
```



Ejercicio N°10

Escribir una subrutina que produzca un delay de 0,5 segundos al invocarla. El propósito de la subrutina es sólo retrasar la ejecución de un programa. **Indicar claramente cuáles son los cálculos empleados para llegar al delay solicitado y cuál es el error cometido por falta de precisión.**

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           15/Sep/20  20:46:58   #
#
#   Source   =   ej10tp2.msa                               #
#   List     =   ej10tp2.lst                               #
#   Object   =   ej10tp2.r07                               #
#   Options  =                                           #
#
#                                                     (c) Copyright IAR Systems 1990 #
#####

1 0000                p68h11
2
3 2000                org      $2000
4 2000 BD3000         jsr      $3000
5 2003 7E2003      fin      jmp      fin
6
7 3000                org      $3000
8 3000 36           psha
9 3001 37           pshb                ;se hace un backup de los registros a modificar
10 3002 3C          pshx
11
12 3003 CE0005       ldx      #$0005       ;crea el contador mayor
13 3006 09          countl  dex
14 3007 270D         beq      endsrt1      ;si es cero el contador mayor va a terminar el programa
15
16 3009 86FF         ldaa     #$FF         ;si no es cero reinicia el contador mediano
17 300B 4A          countm  deca
18 300C 27F8         beq      countl      ;si es cero el contador mediano pasa a decrementar el mayor
19
20 300E C6C3         ldab     #$C3         ;si no es cero reinicia el contador menor
21 3010 5A          counts  decb
22 3011 26FD         bne      counts
23 3013 7E300B       jmp      countm      ;si es cero el contador menor decrementa el mediano
24
25 3016 38          endsrt1 pulx
26 3017 33          pulb                ;se hace restore de los valores guardados
27 3018 32          pula
```



```

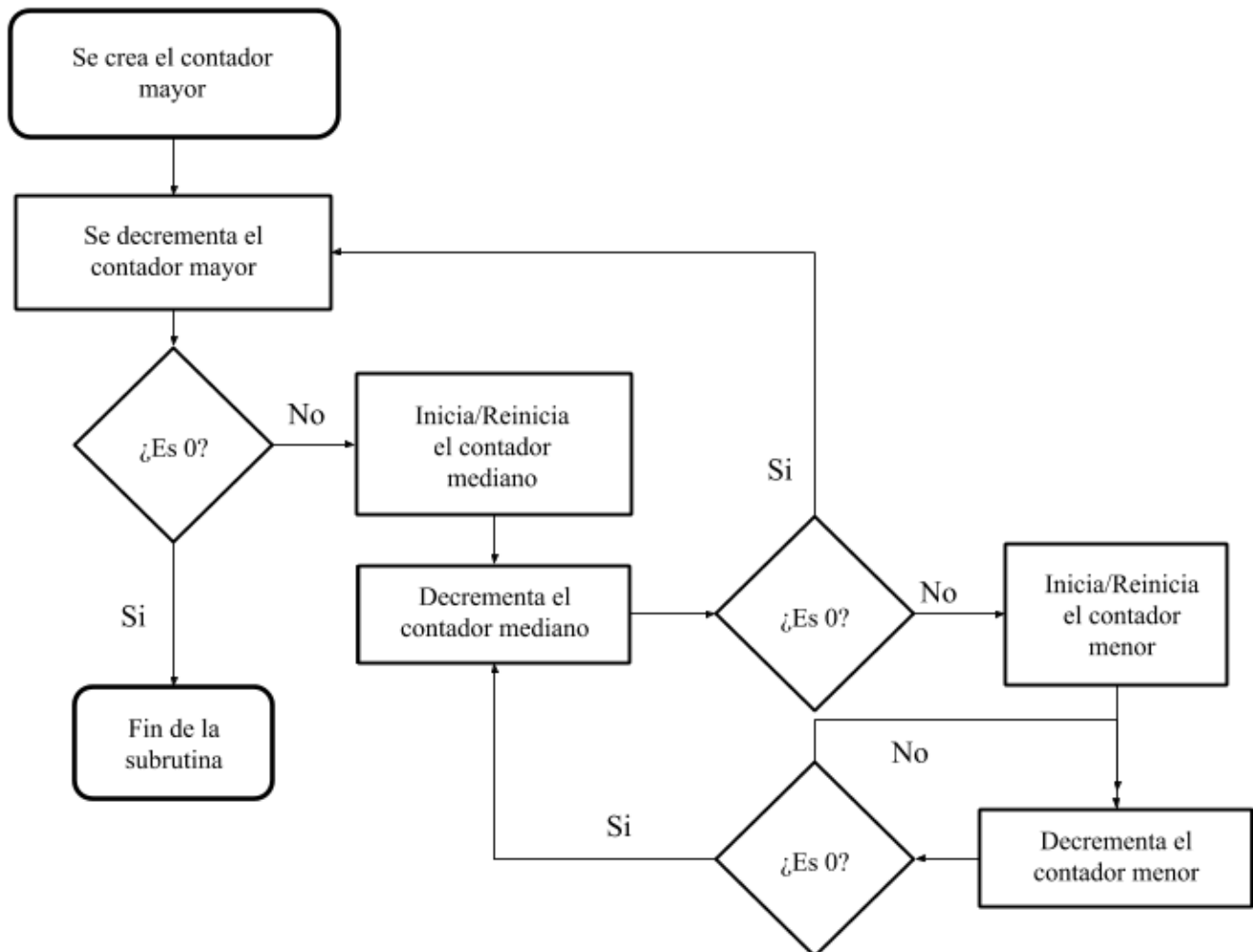
28 3019 39          rts
29
30 8000             org      $8000
31 8000             stack   rmb    30
32 801D             stackP  EQU    *-1
33 801E 8E801D      lds     #stackP
34
35 8021             END

```

```

Errors:  None          #####
Bytes:   35            # ej10tp2 #
CRC:     145A          #####

```



Para calcular qué valor había que asignarle a cada contador tuvimos primero que calcular cuántos ciclos de máquina debía usar la subrutina para llegar a los 0,5 segundos. Como cada ciclo de máquina toma aproximadamente 0,5 microsegundos y cada segundo equivale a 1.000.000 microsegundos, llegamos a que debíamos utilizar 1.000.000 de ciclos de máquina en la subrutina.

Luego de tener una idea de cómo funcionaría el programa, tuvimos que anotar cuántos ciclos tomaba cada instrucción para así poder hacer las cuentas y ver cuántas veces había que repetir cada sección de la subrutina (cantidades que se controlan a partir de los valores ingresados a los contadores).

Desde ahí fue necesario entender cómo el valor de un contador afectaba al resto del programa para conseguir saber exactamente cuantas veces se repetiría cada instrucción. También hubo que observar qué instrucciones se ejecutaban una sola vez sin importar qué modificaciones se hicieran a los contadores.

Por último tuvimos que calibrar bien los valores a ingresar a cada contador para llegar a un número de ciclos totales que se aproximara a 1.000.000 . La subrutina como aparece en el .lst (entre \$3000 y \$3019) utiliza 1.000.849 ciclos de máquina, lo que nos deja extremadamente cerca de los 0,5 segundos que eran el objetivo (0,500425s) si consideramos que cada ciclo de máquina tarda exactamente 0,5 microsegundos en ejecutarse.

Ejercicio N°11

Utilizando los leds que el wookie tiene conectados al puerto A (la dirección del puerto A figura en la hoja de datos del procesador) del HC11 y teniendo en cuenta que solo los bits (3 al 6) del mismo son salidas. Se pide:

- a- Generar un contador binario cuyo valor se refleje en los bits mencionados. El contador deberá contar de 0 a 15 e iniciar la cuenta nuevamente.
- b- Encender un solo led moviendo el mismo de derecha a izquierda (símil “auto fantástico” - https://es.wikipedia.org/wiki/Knight_Rider).
- c- Se deben encender los leds de derecha a izquierda conservando encendido los leds previos.

En todos los casos el intervalo entre secuencias debe ser de 0.5 segundos, se recomienda utilizar la función anterior.

Generar un contador binario cuyo valor se refleje en los bits mencionados. El contador deberá contar de 0 a 15 e iniciar la cuenta nuevamente.

```
#####
#
# Micro Series 6801 Assembler V2.00/DOS          15/Sep/20  22:58:18  #
#
# Source   = 11_a.msa                      #
# List     = 11_a.lst                      #
# Object   = 11_a.r07                      #
# Options  =                               #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

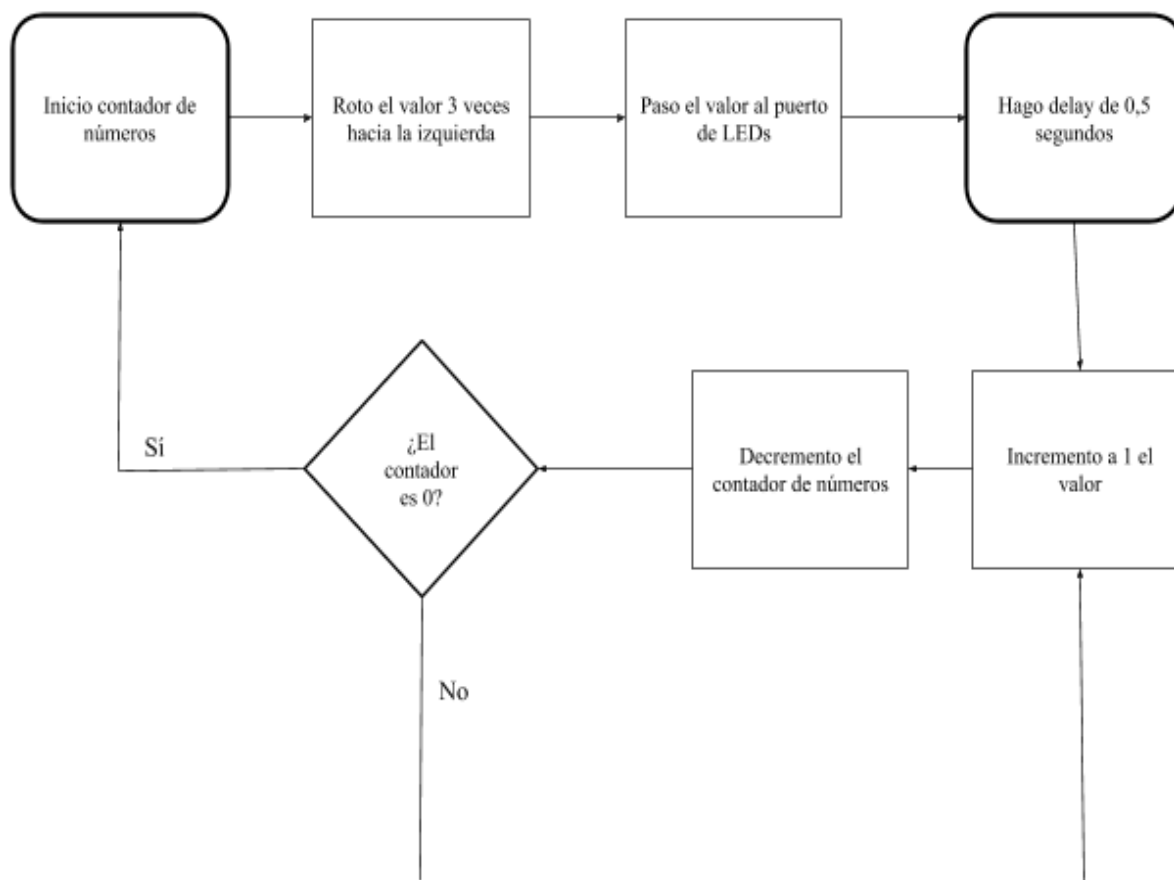
1
2 0000                p68h11                ;Se declara el microprocesador
3
4 *****
5 *                                MAIN PROGRAM                                *
6 *      Éste programa se usa básicamente para poder contar del 0 al 15      *
7 *      de forma repetitiva encendiendo así los leds correspondientes a cada número.  *
8 *****
9 4000                ORG      $4000
10
11 4000 8600          inicio  ldaa    #APAGO          ;inicia un acumulador en 0
12
13 4002 B71000        staa    $1000          ;carga el valor para mostrar apagados los leds del puerto A
14 4005 BD3000        jsr     delay
15 4008 C60F          ldab    #15           ;inicia contador de números
16 400A F74100        stab    $4100          ;pasa el contador a una posición de memoria
17 400D C601          loop1   ldab    #on           ;toma el valor a sumar
18 400F 58            aslb                    ;rota el valor un bit hacia la izquierda 3 veces
19 4010 58            aslb                    ;de forma que así se pueda tener el valor de
20 4011 58            aslb                    ;querido entre los bits 3-6
21 4012 1B            aba                     ;suma el valor anterior
22
23 4013 BD3000        jsr     delay
24 4016 B71000        staa    $1000          ;guarda el valor en una posición de memoria
25 4019 7A4100        dec     $4100          ;decrementa el contador en 1
26 401C 26EF          bne     loop1          ;si el contador no es 0,vuelve a repetir la secuencia
27 401E 27E0          beq     inicio          ;si el contado es 0 repite el todo el programa
28 4020 7E4020        fin     jmp     fin           ;termina el programa
29
30
31 *****
32 *      ÁREA DE DATOS: Aquí están los datos que se usan como variables      *
33 *****
34
35 0001              on      EQU      1
36 0000              APAGO   EQU      00000000b
37
```

```

38
39 *****
40 *                               ÁREA DE LA SUBROUTINA:                               *
41 *   Aquí desarrollamos la subrutina de forma que se genere                               *
42 *   una espera de medio segundo entre el apagado y encendido de los leds *
43 *****
44 3000      delay    org      $3000
45 3000 36          psha
46 3001 37          pshb      ;se hace un backup de los registros a modificar
47 3002 3C          pshx
48
49 3003 CE0005      ldz      #$0005 ;crea el contador mayor
50 3006 09          countl   dex
51 3007 270D          beq      endsrt1 ;si es cero el contador mayor va a terminar la subrutina
52
53 3009 86FF          ldaa     #$FF  ;si no es cero reinicia el contador mediano
54 300B 4A          countm   deca
55 300C 27F8          beq      countl ;si es cero el contador mediano pasa a decrementar el mayor
56
57 300E C6C3          ldab     #$C3  ;si no es cero reinicia el contador menor
58 3010 5A          counts   decb
59 3011 26FD          bne      counts
60 3013 7E300B        jmp      countm ;si es cero el contador menor decrementa el mediano
61
62 3016 38          endsrt1 pulx
63 3017 33          pulb      ;se recuperan los valores guardados
64 3018 32          pula
65 3019 39          rts
66 *****
67 *                               ÁREA DEL STACK: se deja un margen extra                               *
68 *****
69 8000          org      $8000
70 8000          stack    rmb    30
71 801E          stackP   EQU    * -1
72 801E 8E801E      lds      #stackP
73
74
75
76
77 8021          end

Errors:  None          #####
Bytes:   64           # 11_a #
CRC:    9372          #####

```



b- Encender un solo led moviendo el mismo de derecha a izquierda (símil “auto fantástico” - https://es.wikipedia.org/wiki/Knight_Rider).

```

#####
#                                     #
#   Micro Series 6801 Assembler v2.00/DOS           16/Sep/20  10:50:17   #
#                                     #
#   Source   =   11_bene.msa                                           #
#   List     =   11_bene.lst                                           #
#   Object   =   11_bene.r07                                           #
#   Options  =                                                         #
#                                     #
#                                     #
#                                     (c) Copyright IAR Systems 1990 #
#                                     #
#####
  
```

```

1  0000                p68h11        ;declaro el microprocesador
2
3  *****
4  * PROGRAMA PRINCIPAL:El progrma tiene como finalidad encender los leds correspondien- *
5  * tes desde el bit 3 al bit 6 de forma similar al Knight Rider (nosotros adoptamos el *
6  * criterio de que vaya de derecha a izquierda y viceversa.(como se muestra en la serie)*
7  *****
8
9  4000                org    $4000    ;Comienza el programa en una posición de memoria
10
11  4000 C604          inicio    ldab    #CONTA ;Carga el contador en una posición de memoria
12  4002 F74100        stab     $4100    ;y éste contador va  ser el que determine las repeticiones del loop
13
14
15  4005 8600          loop1     ldaa     #APAGO ;Carga el valor para poder
16  4007 B71000        staa     $1000    ;apaga los leds del puerto A
17  400A BD3000        jsr      delay    ;aplica la espera para que se mantenga 0,5 segundos apagado
18  400D 58            aslb                     ;rota el valor del contador de forma que pueda usarlo para prender
                                           ;después los leds que se necesita
19  400E F74150        stab     $4150    ;guarda el valor en una posición de memoria de forma temporal
20  4011 BA4150        oraa     $4150    ;aplica una suma lógica para poder así realizar la secuencia
                                           ;sucesiva que se necesita en el encendido de leds
21  4014 B71000        staa     $1000    ;guarda el valor para encender los leds
22  4017 BD3000        jsr      delay    ;aplica la espera de 0,5 segundos
23  401A 7A4100        dec      $4100    ;decrementa el contador en 1
24  401D 26E6          bne      loop1    ;Si el contador no es 0 vuelve a las instrucciones anteriores
25  401F 8604          ldaa     #CONTA    ;si el contado es 0 vuelve a cargarlo
26  4021 B74100        staa     $4100    ;y así comienza una nueva secuencia pero desde la izquierda hacia
                                           ; a la derecha
27  4024 8600          loop2     ldaa     #APAGO ;apaga los leds de forma que se reinicia el encendido (así comienza
                                           ;todo apagado como la anterior secuencia)
28  4026 F71000        stab     $1000    ;carga el valor para apagarlos
29  4029 BD3000        jsr      delay    ;efectúa la espera de medio segundo en el momento de apagado
30  402C 57            asrb                     ;rota el valor usado para el contador para luego usarlo en el
                                           ;encendido de los leds
31  402D F74150        stab     $4150    ;carga el valor en una memoria para usarlo de forma temporal
32  4030 BA4150        oraa     $4150    ;aplica una suma lógica para poder así realizar la secuencia
                                           ;sucesiva que necesito en el encendido de leds
33  4033 B71000        staa     $1000    ;guarda el valor para que se enciendan los leds
34  4036 BD3000        jsr      delay    ;aplica una espera de medio segundo para mostrarlos encendidos
35  4039 7A4100        dec      $4100    ;decrementa mi contador
36  403C 26E6          bne      loop2    ;Si el contador no es 0 vuelve al loop número 2
37  403E 27C0          beq      inicio    ;si el contador es 0 vuelve a iniciar el programa
38  4040 7E4040        fin       jmp     fin    ;fin del programa
39
40  *****
41  * AREA DE LA SUBROUTINA:Aquí desarrollamos la subrutina *
42  *****
43  3000                delay     org    $3000
44  3000 36            psha

```

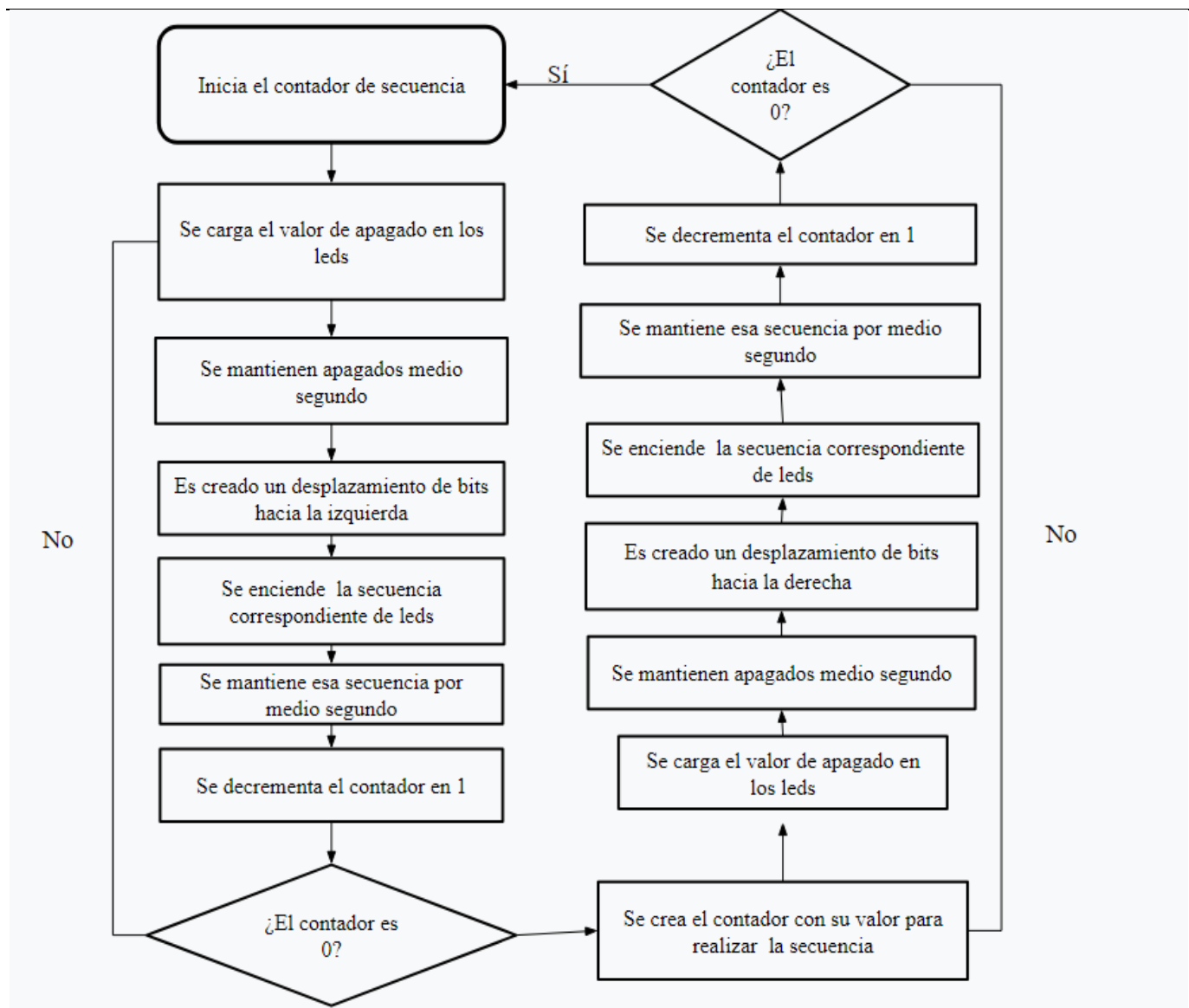
```

45 3001 37          pshb          ;se hace un backup de los registros a modificar
46 3002 3C          pshx
47
48 3003 CE0005      ldx          #$0005 ;crea el contador mayor
49 3006 09          countl      dex
50 3007 270D        beq          endsrtl;si es cero el contador mayor va a terminar la subrutina
51
52 3009 86FF        ldaa         #$FF  ;si no es cero reinicia el contador mediano
53 300B 4A          countm      deca
54 300C 27F8        beq          countl ;si es cero el contador mediano pasa a decrementar el mayor
55
56 300E C6C3        ldab         #$C3  ;si no es cero reinicia el contador menor
57 3010 5A          counts      decb
58 3011 26FD        bne          counts
59 3013 7E300B      jmp          countm ;si es cero el contador menor decrementa el mediano
60
61 3016 38          endsrtl      pulx
62 3017 33          pulb          ;se recuperan los valores guardados
63 3018 32          pula
64 3019 39          rts
66 *****

66      *                      ÁREA DEL STACK:                      *
67      *                      Se reserva espacio del stack con un margen extra                      *
68 *****
68 8000          org          $8000
69 8000          stack      rmb          30
70 801E          stackP     EQU          * -1
71 801E 8E801E    lds          #stackP
72
73 *****
74      *      AREA DE DATOS: Aquí pondré los datos que usaré en mi programa      *
75 *****
76
77 0000          APAGO      EQU          0
78 0004          CONTA      EQU          00000100b
79
80 8021          end

Errors:  None          #####
Bytes:   96           # 11_bene #
CRC:     5722         #####

```



c- Se deben encender los leds de derecha a izquierda conservando encendido los leds previos.

```
#####
#
# Micro Series 6801 Assembler V2.00/DOS          15/Sep/20  22:59:46  #
#
# Source   = 11_c.msa                      #
# List     = 11_c.lst                      #
# Object   = 11_c.r07                      #
# Options  =                               #
#
#                                           (c) Copyright IAR Systems 1990 #
#####
```

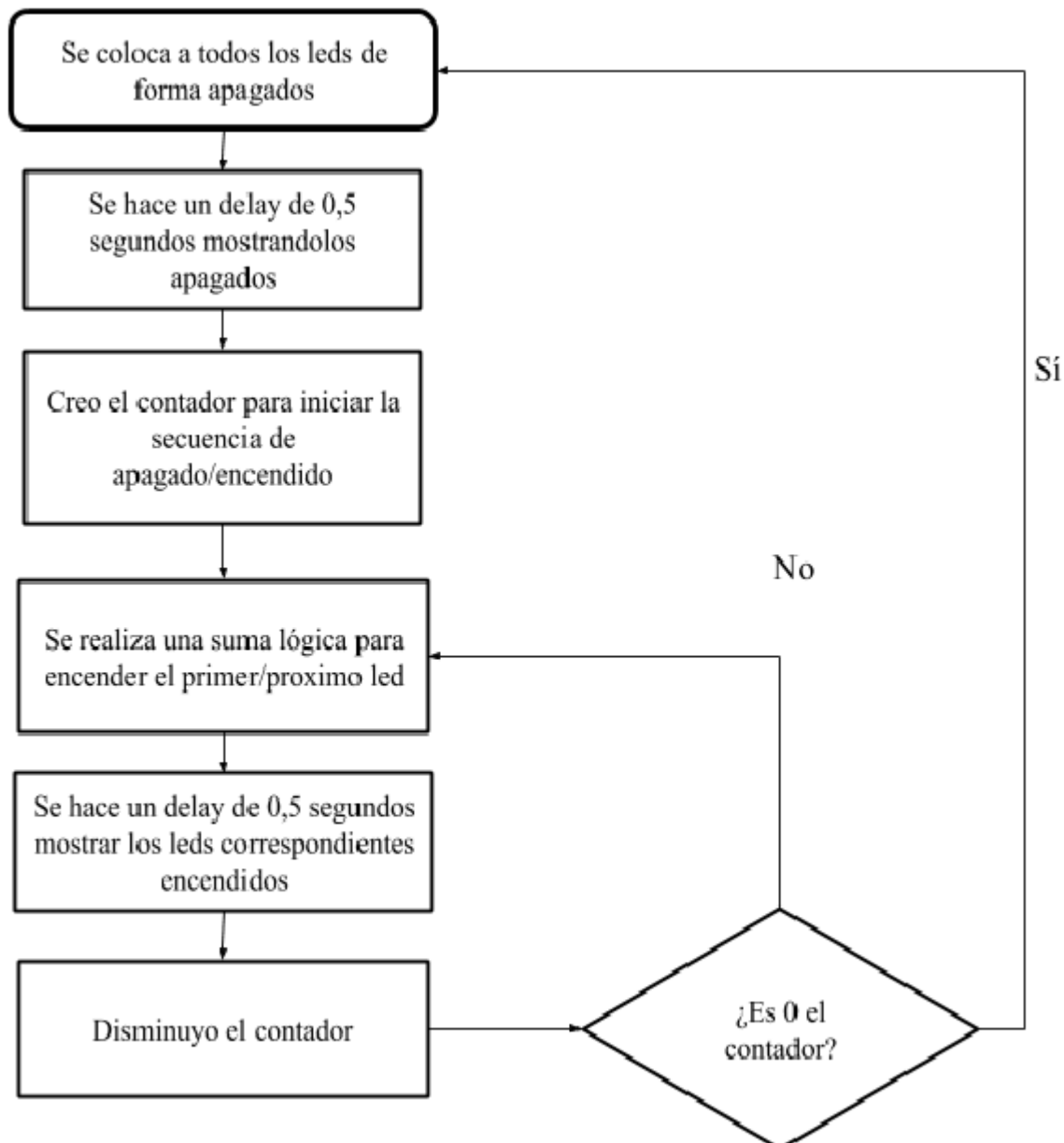
```
1
2 0000          p68h11          ;Declara microprocesador
3
4
5 *****
6 *                               MAIN PROGRAM                               *
7 * :La finalidad de éste programa es poder encender los leds de los      *
8 * bits 3 al 6 de forma que se enciendan y apaguen (con 0,5 segundos de tiempo entre *
9 * encendido y apagado de leds) de derecha a izquierda de forma ordenanda. *
10 *****
11 4000          org      $4000          ;Empieza mi programa en una posición de memoria
12
13 4000 8600      inicio   ldaa     #APAGO          ;carga el valor al puerto de los leds para apagarlos
14 4002 B71000    staa     $1000          ;durante medio segundo
15 4005 BD3000    jsr      delay          ;realiza la espera de medio segundo
16 4008 C604      ldab     #varia          ;inicia contador
17 400A F74100    stab     $4100          ;pasa el contador a una posición de memoria
18
19 400D 58        loop    aslb          ;desplaza el valor un bit hacia la izquierda
20 400E F74150    stab     $4150          ;guarda el valor en una posición de memoria de forma que
21 4011 BA4150    oraa     $4150          ;con una operación ir aumenta el valor en el puerto de leds
22 4014 B71000    staa     $1000          ;para encender dichos leds
23 4017 BD3000    jsr      delay          ;realiza la espera de medio segundo
24
25 401A 7A4100    dec      $4100          ;decrementa el contador en 1
26 401D 26EE      bne     loop          ;Si no es 0 el contador se repite la secuencia anterior
27 401F 27DF      beq     inicio          ;si es 0 vuelve a repetir el programa entero
28 4021 7E4021    fin     jmp     fin          ;se termina el programa
29
30
31 *****
32 *      ÁREA DE LA SUBROUTINA: Aquí se desarrolla la subrutina que genera un      *
33 *      delay de 0,5 segundos entre cada secuencia                               *
34 *****
```

```

34 3000      delay  org    $3000
35 3000 36          psha
36 3001 37          pshb          ;se hace un backup de los registros a modificar
37 3002 3C          pshx
38
39 3003 CE0005      ldx     #$0005          ;crea el contador mayor
40 3006 09          countl dex
41 3007 270D        beq     endsrt1        ;si es cero el contador mayor va a terminar la subrutina
42
43 3009 86FF        ldaa    #$FF          ;si no es cero reinicia el contador mediano
44 300B 4A          countm deca
45 300C 27F8        beq     countl        ;si es cero el contador mediano pasa a decrementar el mayor
46
47 300E C6C3        ldab    #$C3          ;si no es cero reinicia el contador menor
48 3010 5A          counts decb
49 3011 26FD        bne     counts
50 3013 7E300B      jmp     countm        ;si es cero el contador menor decrementa el mediano
51
52 3016 38          endsrt1 pulx
53 3017 33          pulb          ;se recuperan los valores guardados
54 3018 32          pula
55 3019 39          rts
56 301A 39          rts
57 *****
58 *                      ÁREA DEL STACK:                      *
59 *                      Se reerva espacio del stack con un margen extra                      *
60 *****
61 8000          org    $8000
62 8000          stack  rmb    30
63 801E          stackP EQU    * -1
64 801E 8E801E      lds     #stackP
65
66 *****
67 *                      ÁREA DE DATOS:                      *
68 *                      Aquí se presentan los datos a usar en el programa                      *
69 *****
70 0000          APAGO  EQU    0
71 0004          varia EQU    00000100b
72
73
74 8021          end

Errors:  None          #####
Bytes:   66           # 11_c #
CRC:     6D2D         #####

```



Ejercicio N°12

Indicar el estado del stack luego de invocar la instrucción **pshb** en negrita. Indicar la ubicación del stack pointer en el esquema.

	Ldx #\$1000	
Sigo	ldaa 0,X	Sp final
	Psha	AccB
	Inx	“\$1005”
	Cpx #\$1006	10
	Bne Sigo	06
		DRH
	Jsr subrut	DRL
	Swi	“\$1005”
Subrut		“\$1004”
	pshx	“\$1003”
	Tsx	“\$1002”
	Psha	“\$1001”
	pshb	“\$1000”
	rts	

< Sp Inicial

Condiciones de Entrega

Se deberán compilar todos los programas y testearlos mediante el wookie. Se deberá entregar una copia en formato digital de los ejercicios y una copia impresa de los mismos. Se deberán realizar diagramas de flujo para todos los programas. Recordar que los mismos deberán reflejar conceptos o ideas no instrucciones.

Se evaluarán los criterios de programación estipulados por la cátedra. Es de suma importancia documentar correctamente el código agregando comentarios y aclaraciones relevantes.

Programar en forma defensiva, tener en cuenta que parámetros se pueden modificar dentro de las funciones y cuáles no. La eficiencia en la programación será premiada.

Fecha de entrega: 16 de Septiembre